

Pintos 1 Final Report

Team name	한라봉	
Team info.	20210807 이세광	20210750 이명률

1. Current thread system analysis

1) Thread description

Thread는 process 안에서 실행되는 최소 단위의 실행 흐름이다. 프로그램을 수행할 때는 동시에 발생하는 것들을 처리해줘야 하는데, 결국 process를 여러 개 만들어 동시에 수행하도록 해야 한다. 이때 process 전체를 생성하지 말고 기능을 수행하는 부분만 잘라서 생성하고자 thread 개념이 도입되었다. process의 구성 요소는 thread (execution sequence) + address space로 이루어져 있는데, 이 thread를 여러 개로 하고 address space를 그대로 두어 code와 data 영역을 같이 사용하게 하며 stack (user stack + kernel stack)만 따로 만들도록 했다. 각 thread는 자신만의 CPU를 가지고 있다고 착각하며, 사실상 모든 thread끼리 CPU를 공유한다. 이러한 Multi-threading illusion은 thread에 대해 process와 마찬가지로 scheduling함으로써 구현할 수 있다. 또한 Processor 하나에 thread 1개만 실행될 수 있으며, 2개 이상의 thread가 물리적으로 동시에 실행되는 것은 불가능하다. Context switching을 빠르게 하며 여러 thread가 동시에 실행되는 것처럼 환상을 만드는 것이다.



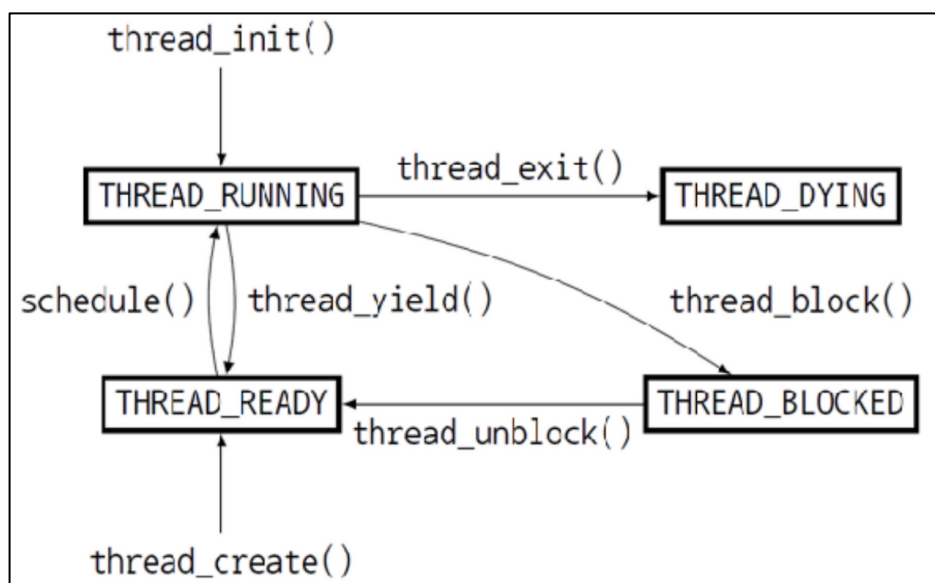
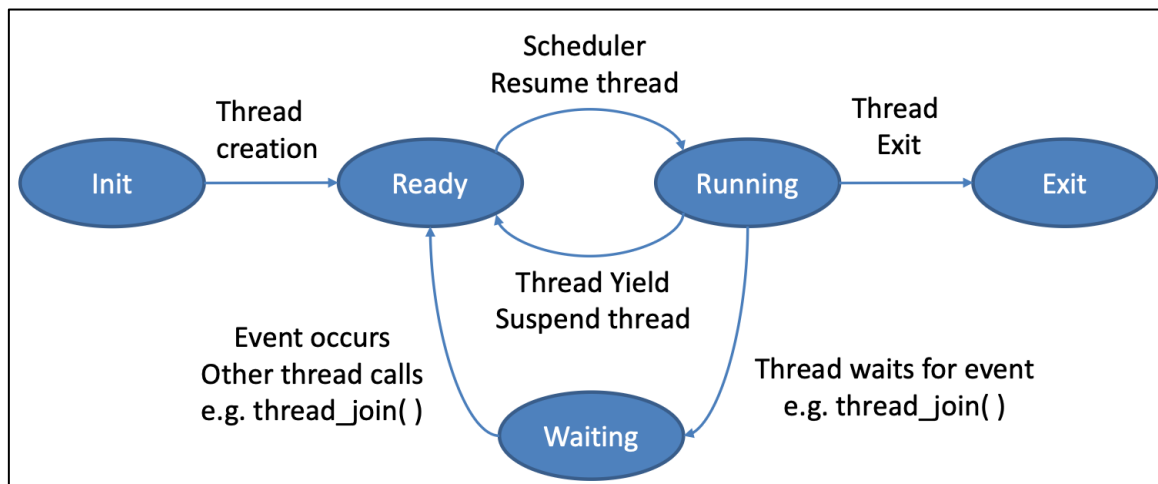
Thread는 2가지 state의 정보를 가진다:

- Shared state: memory (global variable, heap) 내용과 file system 등은 thread들끼리 share한다.
- Private state: program counter, register와 같은 CPU context 정보와 stack의 경우 thread별로 보유한다.

여러 thread들을 create하는 threading 전략은 2가지가 있다:

- 비동기 threading: 부모가 자식 thread를 생성한 후 부모 자신의 실행을 재개하여 부모와 자식 thread가 독립적으로 병행하여 실행되며, data는 shared되지 않는다.
- 동기 threading: 부모 thread가 하나 이상의 자식 thread를 생성하고 자식 thread 모두가 종료될 때까지 기다렸다가 자신의 실행을 재개한다.

Thread의 life cycle을 나타내면 다음과 같다:



2) Pintos 1 skeleton code

```
struct thread
{
    /* Owned by thread.c. */
    tid_t tid; /* Thread identifier. */
    enum thread_status status; /* Thread state. */
    char name[16]; /* Name (for debugging purposes). */
    uint8_t *stack; /* Saved stack pointer. */
    int priority; /* Priority. */
    struct list_elem allelem; /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem; /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir; /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic; /* Detects stack overflow. */
};
```

Pintos 1의 스켈레톤 코드에서 제공된 struct thread는 thread에 대한 모든 정보를 담고 있는 TCB이다.

* TCB (thread control block): thread마다 하나씩 execution state를 보관하는 것 (TCB라는 구조체에 thread switch하기 전 정보를 저장하고 나중에 다시 scheduling)

- tid_t tid – thread를 식별하는 thread 식별자(ID)이다.
- enum thread_status status – thread의 현재 상태 (RUNNING, READY, BLOCKED, DYING)를 나타낸다.
- char name[16] – 디버깅 목적으로 사용되는 thread 이름이다.
- uint8_t *stack – thread의 kernel stack pointer다. Context switch가 이루어질 때 이 stack의 위치가 저장된다.
- int priority – thread의 우선순위로, scheduler가 다음 실행할 thread를 결정하는 데 사용된다.
- struct list_elem allelem – 모든 thread를 연결한 list의 요소이다.
- struct list_elem elem – ready_list, semaphore에서 wait하고 있는 list의 요소이다.
- unsigned magic – stack overflow 감지를 위한 요소이다. Stack이 너무 많이 자라서 struct thread 영역을 침범하면 이 값이 변하게 되고, thread_current() 함수가 이를 감지하여 시스템 오류를 알려준다.

```

/* List of processes in THREAD_READY state, that is, processes
| | that are ready to run but not actually running. */
static struct list ready_list;

/* List of all processes. Processes are added to this list
| | when they are first scheduled and removed when they exit. */
static struct list all_list;

/* Idle thread. */
static struct thread *idle_thread;

/* Initial thread, the thread running init.c:main(). */
static struct thread *initial_thread;

/* Lock used by allocate_tid(). */
static struct lock tid_lock;

```

전역 변수 (in thread.c):

- static struct list ready_list – READY 상태의 thread들을 관리하는 list이다. Scheduler는 이 list에서 다음에 실행할 thread를 꺼내온다.
- static struct list all_list – 시스템에 존재하는 모든 thread를 관리하는 list이다.
- static struct thread *idle_thread – 실행할 thread가 아무것도 없을 때 (ready_list가 비어 있을 때) 실행되는 idle thread를 가리키는 pointer이다.
- static struct thread *initial_thread – init.c의 main()에서 실행되며, initial thread를 나타낸다.
- static struct lock tid_lock – 새로운 tid를 할당할 때 동일한 tid가 2번 이상 할당되는 것을 방지하기 위해 lock을 사용한다.

enum thread_status에 정의된 4가지 상태를 통해 thread의 life cycle이 결정된다.

- THREAD_RUNNING: 현재 CPU를 점유하여 코드를 실행 중인 상태이다.
- THREAD_READY: 언제든지 실행될 수 있지만 대기하는 상태로, ready_list에 포함된다.
- THREAD_BLOCKED: 특정 이벤트(lock 해제, I/O event 등)를 기다리며 잠들어 있는 상태로, ready_list에 포함되지 않는다.
- THREAD_DYING: 실행이 끝나고 소멸을 기다리는 상태이다.

```

void thread_init (void);
void thread_start (void);

void thread_tick (void);
void thread_print_stats (void);

typedef void thread_func (void *aux);
tid_t thread_create (const char *name, int priority, thread_func *, void *);

void thread_block (void);
void thread_unblock (struct thread *);

struct thread *thread_current (void);
tid_t thread_tid (void);
const char *thread_name (void);

void thread_exit (void) NO_RETURN;
void thread_yield (void);

/* Performs some operation on thread t, given auxiliary data AUX. */
typedef void thread_action_func (struct thread *t, void *aux);
void thread_foreach (thread_action_func *, void *);

int thread_get_priority (void);
void thread_set_priority (int);

int thread_get_nice (void);
void thread_set_nice (int);
int thread_get_recent_cpu (void);
int thread_get_load_avg (void);

```

thread 관련 주요 함수:

- void thread_init (void) – thread 시스템을 초기화하는 함수이다.
- void thread_start (void) – idle thread를 생성하며, sema_down()을 통해 idle_thread가 완전히 준비될 때까지 대기한다.
- void thread_tick (void) – timer interrupt가 발생할 때마다 호출된다.
- tid_t thread_create (const char *name, int priority, thread_func *function, void *aux) – 새로운 kernel thread를 생성한다. ready_list에 새 thread를 추가하여 실행 준비 상태로 만들고, 성공하면 새 thread의 id를 반환하고 실패하면 TID_ERROR를 반환한다.
- void thread_block (void) – 현재 실행 중인 thread의 state를 RUNNING에서 BLOCKED로 전환한다.
- void thread_unblock (struct thread *) – BLOCKED된 thread를 READY로 전환한다.
- struct thread *thread_current (void) – 현재 실행 중인 thread의 주소를 반환한다.
- tid_t thread_tid (void) – 현재 실행 중인 thread의 tid를 반환한다.
- const char *thread_name (void) – 현재 실행 중인 thread의 이름을 반환한다.
- void thread_exit (void) NO_RETURN – thread가 종료될 때 호출되며, thread의 state를

DYING으로 전환한다. 또한 `schedule` 함수를 호출하여, context switch를 진행한다.

- `void thread_yield (void)` – 다른 thread에게 CPU를 양도하며, 현재 thread는 `ready_list`에 추가된다.
- `void thread_foreach (thread_action_func *, void *)` – `all_list`에 포함된 모든 thread를 돌면서 `thread_action_func`을 실행한다.
- `void thread_set_priority (int)` – 현재 thread의 priority를 조정한다.
- `int thread_get_priority (void)` – 현재 thread의 priority를 반환한다.
- `static void kernel_thread (thread_func *, void *aux)` – thread func을 실행하며, 함수가 실행 완료되면 `thread_exit()`을 통해 thread를 종료한다.
- `static struct thread *next_thread_to_run (void)` – 다음으로 실행될 thread를 결정한다. 만약 실행 가능한 thread가 없을 경우, `idle_thread`를 반환한다.
- `static bool is_thread (struct thread *)` `UNUSED` – thread가 유효한지 확인한다.
- `static void *alloc_frame (struct thread *, size_t size)` – stack frame을 할당한다.
- `static void schedule (void)` – context switching을 수행한다.

2. Synchronization analysis

Synchronization은 2개 이상의 thread가 동일한 critical resource를 공유하면서 동시에 실행될 때 발생할 수 있는 문제를 다루는 개념이다. (여러 thread가 critical resource에 접근하면 conflict가 발생할 수 있다)

1) Semaphore

Generalized lock으로, semaphore는 non-negative integer value와 wait(), signal() 2개의 operation을 사용한다. 이때 semaphore에서는 각각을 P(), V()라고 부른다. Semaphore->P()는 semaphore 값이 positive가 될 때까지 기다렸다가 1만큼 감소시키고, semaphore->V()는 semaphore 값을 1만큼 증가시키고 P()에 의해 잠든 thread를 깨운다. 또한 P(), V() 모두 atomic operation이다. Pintos 1의 스켈레톤 코드에서는 사용되는 함수와 함수명이 다르다.

```
/* A counting semaphore. */
struct semaphore
{
    unsigned value;           /* Current value. */
    struct list waiters;      /* List of waiting threads. */
};

void sema_init (struct semaphore *, unsigned value);
void sema_down (struct semaphore *);
bool sema_try_down (struct semaphore *);
void sema_up (struct semaphore *);
void sema_self_test (void);
```

- unsigned value – 사용 가능한 공유 자원의 개수를 나타낸다.
- struct list waiters – 공유 자원을 사용하기 위해 대기하는 thread들의 list를 나타낸다.
- void sema_init (struct semaphore *, unsigned value) – semaphore를 초기화한다. Semaphore의 초기 값을 설정하고, semaphore를 기다리는 thread list (waiters)를 초기화한다. 이때 semaphore가 사용 가능하면 1 이상이 할당되고, 사용 가능하지 않으면 0이 할당될 것이다.
- void sema_down (struct semaphore *) – semaphore를 요청한다. 해당 함수는 semaphore의 사용 가능 여부와 관계 없이 호출된다. Semaphore의 value가 0인 경우, thread는 waiters list에 추가되고 block 상태로 전환된다.
- bool sema_try_down (struct semaphore *) – semaphore의 값이 positive이면 value를 1 감소시키고 true를 반환하고, 0인 경우에는 false를 반환한다. 이때 sema_down()과 달리 block되지 않는다.

- void sema_up (struct semaphore *) – 대기 중인 thread가 waiter list에 존재할 경우 첫번째 thread를 깨우고, semaphore 값을 1 증가시킨다.

2) Lock

Lock은 여러 thread나 process가 동시에 공유 자원에 접근할 때 발생할 수 있는 race condition을 방지하기 위해 사용된다. 한 번에 하나만 lock을 acquire할 수 있으며, 다른 thread들은 해당 thread가 lock을 acquire할 동안에는 acquire할 수 없다. Lock acquire는 lock이 free일 때까지 기다리다가 free되면 lock을 가져오고, lock release는 현재 thread가 보유한 lock을 해제하고 다른 waiter를 깨운다. Semaphore와의 차이점은, semaphore는 특정 thread가 down을 수행하고 다른 thread가 up을 수행할 수 있지만, lock은 반드시 lock을 획득한 thread만이 lock을 보유하고 해제할 수 있다. Pintos 1의 스케레톤 코드에서 Lock 관련 메서드의 경우, Semaphore를 구현할 때 사용했던 메서드를 이용하여 구현되어 있다.

```
/* Lock. */
struct lock
{
    struct thread *holder; /* Thread holding lock (for debugging). */
    struct semaphore semaphore; /* Binary semaphore controlling access. */
};

void lock_init (struct lock *);
void lock_acquire (struct lock *);
bool lock_try_acquire (struct lock *);
void lock_release (struct lock *);
bool lock_held_by_current_thread (const struct lock *);
```

- struct thread *holder – lock을 보유하고 있는 thread의 pointer를 저장한다.
- struct semaphore semaphore – thread가 lock을 획득하려 할 때, semaphore 값이 0, 1 중 어떤 값인지에 따라 thread가 대기하거나 lock을 획득하게 된다. 이때 semaphore는 binary로 사용되며 0, 1의 값을 가진다.
- void lock_init(struct lock *) – lock을 초기화한다. Lock을 보유한 thread가 없으며(lock->holder = NULL) lock을 사용 가능한 상태(semaphore를 1로 설정)로 만든다.
- void lock_acquire (struct lock *) – 주어진 lock을 획득하도록 한다. 이때 thread_current()는 실행 중인 thread를 return하는 함수다,.
- bool lock_try_acquire (struct lock *) – lock을 획득하려고 시도한다. 다른 thread가 lock을 이미 보유하고 있는 경우, 대기하지 않고 실패 여부를 반환한다.
- void lock_release (struct lock *) – 현재 thread가 보유한 lock을 해제한다. 이때

semaphore를 1 증가시켜 다른 대기 중인 thread가 lock을 획득할 수 있도록 한다.

- bool lock_held_by_current_thread (const struct lock *) – 현재 실행 중인 thread가 lock을 보유하고 있는지 확인한다.

3) Condition variable

Thread가 lock을 acquire하여 진입한 critical section에서 특정 조건을 만족할 때까지 대기 하도록 하는 방법으로, lock을 잠시 release함과 동시에 자동적으로 sleep하게 된다. (condition variable은 주로 lock과 함께 사용된다) lock release를 통해 다른 thread가 해당 조건을 만족시킬 수 있도록 한다. 이때 lock을 release하는 것과 go to sleep이 atomic하게 이루어져야 한다. Atomic하게 이루어져야 하는 이유를 예시를 들어 설명하면, thread 1이 lock을 release하고 sleep되기 전에 thread 2가 해당 lock을 acquire한 후 조건을 만족시키고 thread 1에 깨우는 signal을 보내는 상황인 경우 thread 1이 아직 sleep되지 않았기 때문에 해당 signal이 무시되고 thread 1은 그 이후에 sleep되어 영원히 깨어나오지 못하는 문제가 발생할 수 있기 때문이다. Pintos 1의 스켈레톤 코드에서 condition variable 관련 메서드의 경우, semaphore 및 lock을 구현할 때 사용했던 메서드를 이용하여 구현되어 있다.

```
/* Condition variable. */
struct condition
{
    struct list waiters;      /* List of waiting threads. */
};

void cond_init (struct condition *);
void cond_wait (struct condition *, struct lock *);
void cond_signal (struct condition *, struct lock *);
void cond_broadcast (struct condition *, struct lock *);
```

- struct list waiters – 대기 중인 thread 목록으로, 특정 조건에 대해 대기 중인 thread들은 waiters list에 추가된다. 조건이 충족되면, 해당 threads들은 list에서 제거된다.
- void cond_init (struct condition *) – waiters (condition variable)를 초기화한다.
- void cond_wait (struct condition *, struct lock *) – lock release → go to sleep → 특정 조건이 만족되었을 때 lock을 reacquire하도록 한다.
- void cond_signal (struct condition *, struct lock *) – 대기 중인 thread 중 하나에 신호를 보내어 해당 thread를 깨운다.
- void cond_broadcast (struct condition *, struct lock *) – 대기 중인 모든 thread들을 깨운다.

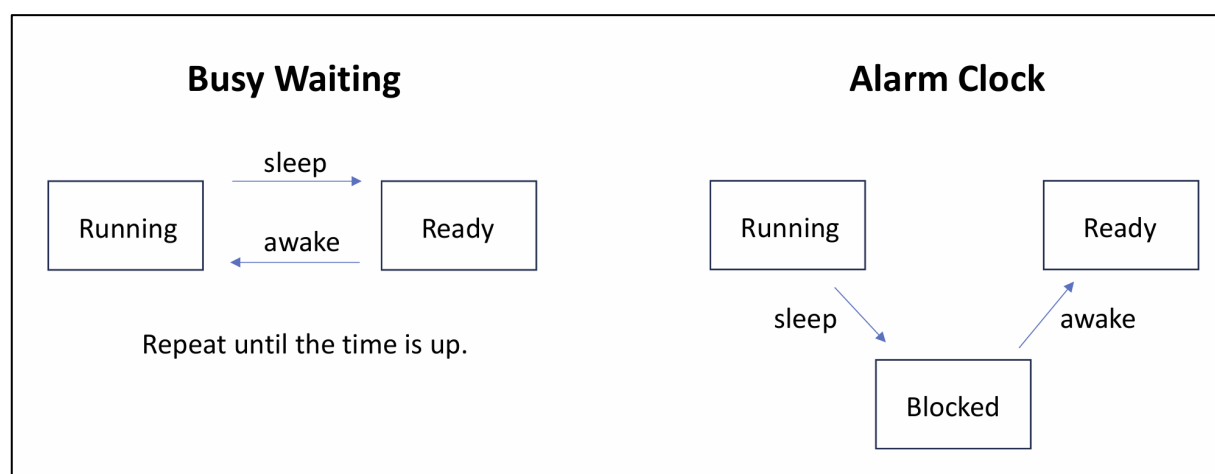
3. Goal of Pintos 1 & Pintos 1 Implementation

1) Alarm clock

```
1. /** Sleeps for approximately TICKS timer ticks. Interrupts must
2.     be turned on. */
3. void
4. timer_sleep (int64_t ticks)
5. {
6.     int64_t start = timer_ticks ();
7.
8.     ASSERT (intr_get_level () == INTR_ON);
9.     while (timer_elapsed (start) < ticks)
10.         thread_yield ();
11. }
```

Pintos 1의 스켈레톤 코드에서 `timer_sleep()`은 `ticks`(thread가 잠들어 있어야 하는 시간)를 파라미터로 받아 해당 thread의 실행을 지연시킨다. `timer_ticks()`를 통해 thread가 실행된 시점을 `start`로 기록하고, `timer_elapsed()`를 통해 현재까지 경과한 시간을 확인한다. 그리고 이 값이 `ticks`보다 작은 동안에는 `thread_yield()`를 호출하여 CPU를 다른 thread에 yield한다. 이 방식은 while문의 조건이 참인 동안 thread는 계속 반복문을 돌며 `thread_yield()`를 호출하기 때문에, busy waiting으로 구현되어 있어 비효율적이다.

현재 스켈레톤 코드에서는 ready list에서 스레드를 하나 꺼내서 이 스레드가 잠들어야 되는 시간보다 경과된 시간이 지났는지 확인을 하고 그렇지 않았다면 다시 ready list 맨 뒤로 삽입한다. 게다가 현재 ready list는 앞선 `next_thread_to_run()`에서 봤듯이 FIFO로 구현되어 있다. 즉, 단순 Queue 구조로 이루어져 있다. 그렇다면 경과된 시간에 상관없이 삽입된 순서대로 계속 큐 맨 앞에서 꺼내 와서 해당 조건문을 확인하게 된다. 이는 매우 비효율적인 방식으로 핀토스 매뉴얼에서도 현재 방식은 실시간으로 동작하는 시스템에서만 좋다고 설명하고 있다.



이러한 Busy waiting을 대신하여, Pintos 1에서는 sleep 및 wake up 기반의 alarm clock 방식으로 다시 구현해야 한다. 즉, 별도의 list를 관리하여 아직 지정된 시간이 지나지 않았으면 block 상태로 만들어 관리하여 sleep하게 만들고 시간이 지나면 알람 시계처럼 일

어나게 만들어 Ready list로 만드는 구조로 개선해야 할 필요가 있다.

Initial Design:

기본적으로 thread를 ready_list에 넣어 반복 실행시키는 기존의 방식 대신, sleep_list라는 별도의 list를 정의하여 thread를 block 상태로 관리한다. 이후 지정된 시간이 지나면 sleep_list에 있는 thread들을 깨워 ready_list로 이동시켜 다시 scheduling할 수 있도록 한다.

코드 설계를 위해 디자인한 변수 및 함수는 다음과 같다 (변수 및 함수는 Pintos 1 스케레톤 코드를 참고하여 거시적인 관점에서 고려하였다):

Variables:

- static struct list sleep_list – 잠들어 있는 thread들을 관리하는 list이다. list.c에 정의된 기본 list 연산을 활용하여 관리한다. wakeup_tick 기준으로 오름차순 정렬하여 효율적인 삽입을 가능하도록 한다.
- int64_t wakeup_tick – 해당 thread가 깨어나야 하는 시스템 tick 수를 저장한다. timer_sleep() 호출 시 현재 tick + 지정한 대기 tick 수를 저장하도록 구현한다.

Functions:

- timer_init() – 타이머 초기화 시 sleep_list도 같이 초기화하도록 수정한다.
- timer_sleep(int64_t ticks) - 호출 thread의 wakeup_tick을 현재 tick + 대기 tick 수로 계산하여 설정하고, thread를 sleep_list에 wakeup_tick 기준 오름차순 정렬해 삽입한다. 또한 thread를 block 상태로 전환하여 CPU 사용권을 포기하도록 한다.
- timer_interrupt() – sleep_list에서 wakeup_tick이 지나서 깨어나야 할 thread를 탐색하여, list에서 제거 후 thread_unblock()을 호출한다. 이를 통해 해당 thread가 ready_list로 이동해 다시 실행 준비 상태가 된다.
- wake_less() – sleep_list의 간단한 비교 함수로 wakeup_tick 값에 따라 list 삽입 순서를 정의하기 위해 추가하였다. list_insert_ordered()에 전달되어 쓰일 예정이다.

1-1) Implementation:

```
1. // threads/thread.h
2.
3. struct thread{
4.     int64_t wakeup_tick;
5. };
6.
```

struct thread에 thread가 깨어날 시간을 저장할 멤버를 추가해야 하므로 int64_t wakeup_tick을 struct thread의 멤버로 선언했다.

A. 전역 변수 추가

```
1. // devices/timer.c
2.
3. static struct list sleep_list;
```

잠자는 thread들을 관리할 list를 static 전역 변수로 선언했다.

B. timer_init() 함수 수정

```
1. // devices/timer.c
2.
3. void
4. timer_init (void)
5. {
6.     pit_configure_channel (0, 2, TIMER_FREQ);
7.     intr_register_ext (0x20, timer_interrupt, "8254 Timer");
8.
9.     /* sleep_list 초기화 */
10.    list_init(&sleep_list);
11. }
```

timer_init()에서 새로 추가한 sleep_list를 초기화해야 한다.

C. timer_sleep() 재구현 및 wake_less() 커스텀 함수 구현

```
1. // devices/timer.c
2.
3. void
4. timer_sleep (int64_t ticks)
5. {
6.     if (ticks <= 0) {
7.         return;
8.     }
9.
10.    enum intr_level old_level = intr_disable ();
11.
12.    struct thread *cur = thread_current ();
13.
14.    cur->wakeup_tick = timer_ticks () + ticks;
15.
16.    list_insert_ordered (&sleep_list, &cur->elem, wake_less, NULL);
17.
18.    thread_block ();
19.
20.    intr_set_level (old_level);
21. }
```

```
1. // devices/timer.c
2.
3. static bool
4. wake_less (const struct list_elem *a, const struct list_elem *b, void *aux UNUSED)
5. {
6.     const struct thread *thread_a = list_entry (a, struct thread, elem);
7.     const struct thread *thread_b = list_entry (b, struct thread, elem);
8.     return thread_a->wakeup_tick < thread_b->wakeup_tick;
9. }
10.
```

현재 thread가 깨어나야 할 시간을 계산하여 저장하고 thread를 sleep_list에 추가하여 block 상태로 만든다. 나중에 unblock되면 다시 실행 재개를 한다. 이때 sleep_list를 항상 wakeup_tick 기준 오름차순으로 유지하기 위해, wake_less() 커스텀 비교 함수와 이미 구

현된 `list_insert_ordered()`를 이용했다. 이렇게 구현하여 `sleep list`에서 스레드를 꺼낼 때 가장 일찍 일어나야 하는 스레드를 빠르게 꺼낼 수 있도록 효율적으로 만들었다.

D. `timer_interrupt()` 함수 수정

```
1. // devices/timer.c
2.
3. static void
4. timer_interrupt (struct intr_frame *args UNUSED)
5. {
6.     ticks++;
7.     thread_tick ();
8.
9.     if(list_empty (&sleep_list)) {
10.         return;
11.     }
12.
13.     struct list_elem *e = list_begin (&sleep_list);
14.
15.     while(e != list_end (&sleep_list)) {
16.         struct thread *t = list_entry (e, struct thread, elem);
17.
18.         if(ticks >= t->wakeup_tick) {
19.             struct list_elem *next = list_next (e);
20.             list_remove (e);
21.             thread_unblock (t);
22.             e = next;
23.         } else {
24.             break;
25.         }
26.     }
27. }
```

기존에는 주기적으로 호출되어 시스템 tick을 증가시키는 기능 밖에 없었다. 이제는 `sleep list`를 만들었기 때문에 알람시계 역할이 필요하다. 일정 시간이 지난 후 깨워야 할 thread를 관리하도록 한다. `Sleep list`를 순회하며 깨워야 될 스레드가 있으면 `unblock`하여 `ready` 상태로 만든다. 이때 `sleep list`를 `wakeup_tick` 기준 오름차순으로 유지하는 로직을 적용했기 때문에, `while`문을 순회하면서 깨워야 할 thread가 있을 경우 `unblock`한 뒤 `break`를 사용했다.

2) Priority scheduling

기존 `thread_tick()` 함수에서는 `thread_ticks`가 `TIME_SLICE`(기본 4 tick)에 도달하면, `preemption`이 필요하다는 것을 표시하여 `interrupt`가 끝날 때 `scheduling`이 일어나게 한다. 따라서 기존 `thread_tick()` 함수를 통해 `round robin` 방식으로 구현되어 있음을 확인할 수 있다.

Pintos 1의 스케줄론 코드에서는 `round-robin (RR) scheduler`가 구현되어 있는데, 이러한 `scheduling` 방식은 각 thread의 우선 순위를 고려하지 않으며 (정해진 시간 할당량만큼 process를 할당하여 시간이 지나면 currently running process는 ready queue의 가장 마지막에 가서 재할당을 기다린다) `priority scheduling` 과정에서 `priority inversion problem` (높

은 우선순위의 프로세스가 낮은 우선순위의 프로세스로 인해 수행이 block되어 있는 상태)이 발생할 수 있다.

Pintos 1에서는 현재 구현되어 있는 round-robin scheduler를 priority scheduler로 바꿔야 한다. 이때 다음의 priority scheduling rule을 따라야 한다:

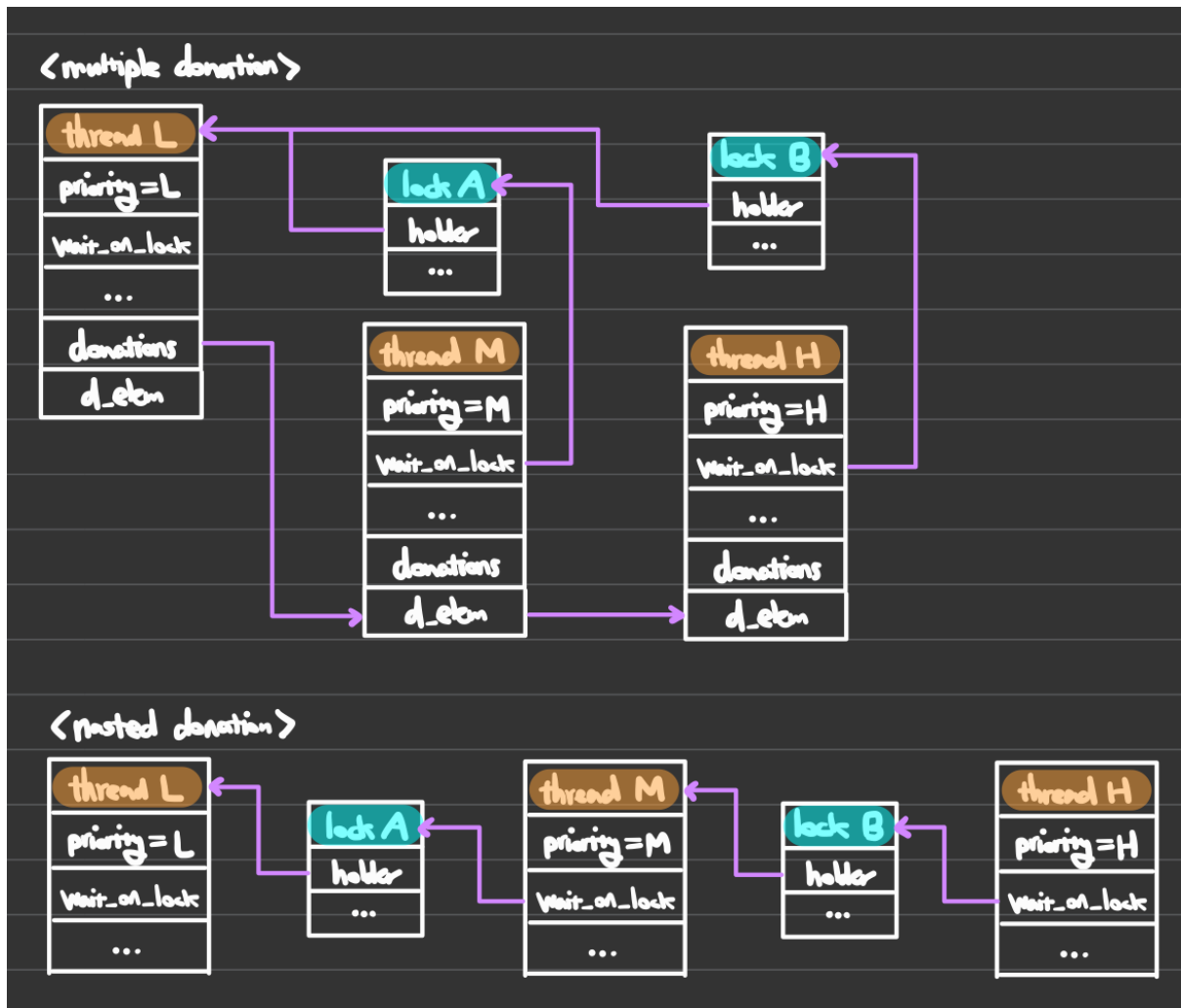
- Threads가 ready to run되면, highest priority를 가진 thread가 먼저 schedule되어야 한다.
- Running thread보다 higher priority를 가진 thread가 ready되면, 현재 running thread는 즉시 CPU를 yield해야 한다.
- Threads가 synchronization 대기 중일 경우, highest priority를 가진 thread가 먼저 awake되어야 한다.
- Thread가 자신의 우선순위를 낮춰 더 이상 가장 높은 우선순위를 가지지 않게 되면, 해당 thread는 즉시 CPU를 yield해야 한다.

Initial Design:

설계의 핵심은 다음의 두 가지이다:

(1) ready_list를 우선순위 queue로 만든다. 우선순위 queue로 만들기 위해, ready_list를 항상 우선순위 순으로 정렬된 상태를 유지하도록 설계하려 한다.

(2) 우선순위 기부(priority donation)를 구현하는 것이다. priority scheduling의 한 가지 문제는 우선순위 역전(priority inversion)이다. 예를 들어 설명해보자면, 높은, 중간, 낮은 우선순위를 가진 thread를 각각 H, M, L이라고 해보자. 만약 H가 L이 점유한 lock을 기다려야 하고 M이 ready queue에 있다면, $M \rightarrow L \rightarrow H$ 의 순으로 동작하게 되어 H가 M보다 우선순위가 높음에도 불구하고 M이 먼저 실행되는 문제가 발생할 수 있다. 이 문제에 대한 부분적인 해결책은, L이 lock을 보유하고 있는 동안 H가 자신의 우선순위를 L에게 donate하고 L이 lock을 해제하여 H가 lock을 다시 획득하면 donate를 철회하는 것이다. 즉, lock을 기다리는 thread가 자신의 높은 priority를 lock을 보유한 낮은 우선순위의 thread에게 일시적으로 빌려주는 priority donation을 통해, priority inversion을 해결할 수 있다. 이를 위해, Priority donation이 필요한 모든 상황을 고려해야 한다. 또한 여러 thread가 하나의 thread에게 priority를 기부하는 다중 기부(multiple donations)를 처리해야 하며, 중첩된 기부(nested donation)도 처리해야 한다. 예를 들어 H가 M이 보유한 lock을 기다리고 M은 L이 보유한 lock을 기다린다면, M과 L은 모두 H의 priority로 올라가야 한다.



코드 설계를 위해 디자인한 변수 및 함수 (+ 기존 스켈레톤 코드에서 수정하려 하는 변수 및 함수)는 다음과 같다 (변수 및 함수는 Pintos 1 스켈레톤 코드를 참고하여 거시적인 관점에서 고려하였다):

Variables:

- int base_priority – thread가 원래 가지고 있던 기본 우선순위로, priority donation을 받기 전의 값이다.
- struct lock *waiting_lock – 현재 thread가 기다리고 있는 lock을 가리키는 pointer이다.
- struct list donations – 이 thread에 priority를 donate한 다른 thread들의 list이다. Priority donation 관리에 사용된다.
- struct list_elem donation_elem – 다른 thread의 donation list에 포함될 때 쓰이는 list 요소이다.
- struct list waiters – 해당 lock을 가리키는 thread들을 priority 순서대로 정렬하여 저장하는 list이다.

Functions:

- thread_unblock(struct thread *t) – ready_list에 들어갈 때 priority 순으로 정렬하기 위해, 기존 list_push_back()에서 list_insert_ordered()로 수정하려 한다.
- thread_yield() – 위와 마찬가지로 실행 중인 thread에 CPU를 양보할 때, ready_list에 priority 순서대로 삽입하도록 수정하려 한다.
- sema_down(), cond_wait() – lock이나 조건 변수 대기 시 priority에 따라 대기 list에 삽입하도록 수정한다.
- cond_signal() – 대기 중인 thread 중 가장 높은 priority를 가진 thread를 깨우도록 수정한다.
- thread_recalculate_priority(struct thread *t) – 기본 priority와 donate받은 priority 중 최고값을 선택해 thread 우선순위를 갱신한다.
- thread_donate_priority(struct thread *holder) – 높은 우선순위 thread가 lock 대기 시 낮은 우선순위 lock 소유자에게 priority donation을 수행하고, donation은 lock 대기 중인 여러 thread에 걸쳐 연쇄적으로 전파되도록 한다.
- lock_acquire(struct lock *lock) – lock 점유 중일 경우 현재 thread는 waiting_lock에 lock을 기록하고, lock 소유자에 priority donation 수행 후 대기하도록 구현하려 한다.
- thread_remove_donations_for_lock(struct lock *lock) – lock이 해제되면 관련된 priority donation 기록을 삭제하도록 한다.
- lock_release(struct lock *lock) – lock 해제 및 donation 목록 정리, priority 재계산 후 가장 높은 priority thread를 깨운다.
- test_max_priority() – ready_list 맨 앞의 가장 높은 priority thread가 현재 실행 thread보다 우선순위가 높으면 CPU를 양보하도록 한다.

2-1) Implementation:

이전 Design Report에서 명시한 핵심 사항 2가지에 더하여, 우선순위가 높은 thread가 Ready에 있으면, 바로 선점해야 한다는 규칙을 추가 고려해야 한다. Priority scheduler 구현, 선점(항상 priority가 높은 thread가 CPU를 점유하는 것), priority donation 이 3가지에 주의하며 구현하였다.

먼저, 자료 구조이다.

```
1. // threads/thread.h
2.
3. struct thread{
4.     int base_priority;           /* 기부 받기 전의 원래 우선순위 */
5.     struct lock *waiting_lock;  /* 현재 이 스레드가 기다리고 있는 lock */
```



```

6.     struct list donations;                /* 이 스레드에게 우선순위를 기부한 스레드들의 리스트 */
7.     struct list_elem donation_elem;      /* 다른 스레드의 donations 리스트에 들어갈 때 사용할
element */
8. };

```

priority scheduling과 donation에 필요한 정보를 struct thread에 추가하기 위해 위의 코드와 같이 데이터 구조를 작성했다. 기존 thread에 이미 선언된 priority는 현재 해당 스레드의 우선순위를 표현하도록 하였다. 그리고 priority가 바뀌는 것을 대비하여, base_priority를 따로 선언하여 기부 받기 전 원래 우선순위를 저장해두도록 했다. waiting_lock은 현재 이 스레드가 기다리고 있는 lock, 그리고 이 스레드에게 우선순위를 기부한 스레드들의 리스트도 저장해 두도록 했다.

```

1. // threads/synch.h
2.
3. struct lock {
4.     struct thread *holder;    /* Thread holding lock (for debugging). */
5.     struct semaphore semaphore; /* Binary semaphore controlling access. */
6.
7.     struct list waiters;
8. };

```

추가로 struct lock에 해당 lock을 기다리는 thread list를 만들어주었다.

A. ready_list를 priority queue로 만들기

현재 ready_list는 일반적인 queue로 FIFO 구조를 가지고 있다. 이 구조에 대해, priority가 높은 thread가 높은 thread가 항상 먼저 나올 수 있도록 수정해야 한다. 그때그때 Queue를 순회하며, 가장 높은 우선순위의 thread를 꺼내는 방법도 있지만, 가장 좋은 방법은 ready_list를 항상 priority 순의 정렬된 상태로 유지하는 것이다. (전자의 경우 pop을 할 때마다 매번 list를 확인해야 하기 때문에 보다 효율적인 후자의 방법을 택했다) 이를 위해, 현재 thread_unblock(), thread_yield() 두 함수 내부에 있는 list_push_back(&ready_list) 코드를 list_insert_ordered()로 변경했다.

```

1. // threads/thread.c
2.
3. void
4. thread_unblock (struct thread *t)
5. {
6.     enum intr_level old_level;
7.
8.     ASSERT (is_thread (t));
9.
10.    old_level = intr_disable ();
11.    ASSERT (t->status == THREAD_BLOCKED);
12.    list_insert_ordered (&ready_list, &t->elem, priority_less, NULL);
13.    t->status = THREAD_READY;
14.    intr_set_level (old_level);
15. }

```

```

1. void
2. thread_yield (void)
3. {
4.     struct thread *cur = thread_current ();
5.     enum intr_level old_level;
6.
7.     ASSERT (!intr_context ());

```

```

8.
9.  old_level = intr_disable ();
10. if (cur != idle_thread)
11.     list_insert_ordered (&ready_list, &cur->elem, priority_less, NULL);
12.  cur->status = THREAD_READY;
13.  schedule ();
14.  intr_set_level (old_level);
15. }

```

synch.c에서도 thread를 대기시키는 곳이 있는데, 이 부분도 마찬가지로 모두 수정해야 한다. sema_up(), sema_down(), cond_wait(), cond_signal()이다. (lock_acquire(), lock_release())의 경우 sema_up(), sema_down()을 이용하여 구현되어 있으므로 따로 처리해줄 필요가 없으므로 수정하지 않았다)

```

1. // threads/synch.c
2.
3. void
4. sema_down (struct semaphore *sema)
5. {
6.     enum intr_level old_level;
7.
8.     ASSERT (sema != NULL);
9.     ASSERT (!intr_context ());
10.
11.    old_level = intr_disable ();
12.    while (sema->value == 0)
13.    {
14.        list_insert_ordered(&sema->waiters, &thread_current()->elem, priority_less, NULL);
15.        thread_block ();
16.    }
17.    sema->value--;
18.    intr_set_level (old_level);
19. }
20.

```

```

1. void
2. sema_up (struct semaphore *sema)
3. {
4.     enum intr_level old_level;
5.
6.     ASSERT (sema != NULL);
7.
8.     old_level = intr_disable ();
9.
10.    if (!list_empty (&sema->waiters))
11.    {
12.        list_sort(&sema->waiters, priority_less, NULL);
13.        thread_unblock (list_entry (list_pop_front (&sema->waiters),
14.                                     struct thread, elem));
15.    }
16.    sema->value++;
17.    test_max_priority();
18.    intr_set_level (old_level);
19. }
20.

```

semaphore에서는 하나의 공유 자원을 사용하기 위해 여러 thread가 sema_down 되어 대기하고 있다. 한편, 이 공유 자원을 쓰던 thread가 마치고 sema_up할 때 어떤 thread가 가장 먼저 차지할지를 결정해야 하는 것이 여기서의 문제이다.

기존 sema_down에서는 사용 가능한 자원이 없다면, 맨 뒤로 push_back을 했다면, 이제는 list_insert_ordered()에 priority_less(priority를 내림차순으로 비교하는 함수)를 더해 내

림차순으로 삽입하여 우선순위가 높다면 앞쪽으로 낮다면 뒤쪽으로 배치하도록 한다. 기존 sema_up에서는 FIFO로 앞쪽에서 꺼내 오기만 했는데, 이를 유지는 하되 항상 큐가 우선순위대로 정렬하도록 하기 위해 list_sort()를 적용했다.

```

1. // threads/synch.c
2.
3. bool conditional_var_comparator(struct list_elem *a, struct list_elem *b, void *aux){
4.
5.     struct semaphore_elem *semaphore_one = list_entry(a, struct semaphore_elem, elem);
6.     struct semaphore_elem *semaphore_two = list_entry(b, struct semaphore_elem, elem);
7.
8.     struct thread *s_one= list_entry(list_front(&semaphore_one->semaphore.waiters), struct
thread, elem);
9.     struct thread *s_two= list_entry(list_front(&semaphore_two->semaphore.waiters), struct
thread, elem);
10.
11.     if(s_one->priority > s_two->priority)
12.         return true;
13.     else return false;
14. }
15.

```

```

1. void
2. cond_wait (struct condition *cond, struct lock *lock)
3. {
4.     struct semaphore_elem waiter;
5.
6.     ASSERT (cond != NULL);
7.     ASSERT (lock != NULL);
8.     ASSERT (!intr_context ());
9.     ASSERT (lock_held_by_current_thread (lock));
10.
11.     sema_init (&waiter.semaphore, 0);
12.     list_insert_ordered(&cond->waiters, &waiter.elem, priority_less, NULL);
13.     lock_release (lock);
14.     sema_down (&waiter.semaphore);
15.     lock_acquire (lock);
16. }

```

```

1. // threads/synch.c
2.
3. void
4. cond_signal (struct condition *cond, struct lock *lock UNUSED)
5. {
6.     ASSERT (cond != NULL);
7.     ASSERT (lock != NULL);
8.     ASSERT (!intr_context ());
9.     ASSERT (lock_held_by_current_thread (lock));
10.
11.     if (!list_empty (&cond->waiters)) {
12.         list_sort(&cond->waiters, (list_less_func *) &conditional_var_comparator, NULL);
13.
14.         sema_up (&list_entry (list_pop_front (&cond->waiters),
15.                                     struct semaphore_elem, elem)->semaphore);
16.     }
17. }

```

이번에는 condition variable이다. 여기 condition variable도 semaphore와 동작은 유사하다. cond_wait – sema_down / cond_signal – sema_up. 구조적으로는 semaphore에서는 waiter list에 thread를 직접 추가했지만, condition variable에서는 semaphore를 waiters list에 넣는다. 다만 동작은 유사하여 각각 list_insert_ordered(), list_sort()를 추가하여 주었다.

```

1. // threads/thread.c

```

```

2.
3. bool
4. priority_less (const struct list_elem *a, const struct list_elem *b, void *aux UNUSED)
5. {
6.     const struct thread *thread_a = list_entry (a, struct thread, elem);
7.     const struct thread *thread_b = list_entry (b, struct thread, elem);
8.     return thread_a->priority > thread_b->priority;
9. }
10.

```

위 함수가 list_insert_ordered, list_sort()에서 우선순위 비교에 쓰였다. Priority 기준으로 내림차순으로 정렬하는데 쓰인다.

B. Preemption(선점)

더 높은 priority의 thread가 실행 가능 상태가 되면, 현재 실행 중인 thread는 즉시 CPU를 양보해야 한다. 즉, 우선순위가 높은 thread가 선점해야 한다.

```

1. // threads/thread.c
2.
3. static void
4. test_max_priority (void)
5. {
6.     if (list_empty(&ready_list))
7.         return;
8.
9.     struct thread *next_thread = list_entry(list_front(&ready_list), struct thread, elem);
10.    if (thread_current()->priority < next_thread->priority)
11.    {
12.        thread_yield();
13.    }
14. }
15.

```

ready_list에 있는 가장 priority가 높은 thread가 현재 실행 중인 thread보다 더 높은 priority를 가지는 경우, CPU를 양보하도록 하는 함수를 위와 같이 구현했다.

현재 실행 중인 thread의 priority가 바뀌는 상황은 총 두 가지이다. thread_create(), thread_set_priority()이며, 이 두 곳에 test_max_priority()를 추가하여, running thread와 ready_list의 가장 앞에 있는 thread의 priority를 비교했다.

```

1. // threads/thread.c
2.
3. tid_t
4. thread_create (const char *name, int priority, thread_func *function, void *aux) {
5.     /* Stack frame for switch_threads(). */
6.     sf = alloc_frame (t, sizeof *sf);
7.     sf->eip = switch_entry;
8.     sf->ebp = 0;
9.
10.    /* Add to run queue. */
11.    thread_unblock (t);
12.    test_max_priority();
13.
14.    return tid;
15. }
16.

```

```

1. // threads/thread.c
2.
3. void
4. thread_set_priority (int new_priority)

```

```

5. {
6.     if (thread_mlfqs) {
7.         return;
8.     }
9.
10.    thread_current ()->base_priority = new_priority;
11.    thread_recalculate_priority(thread_current()); // 기부 상태를 고려하여 실질 우선순위 재계산
12.    test_max_priority(); // 선점 확인
13. }

```

이 함수는 이제 thread의 기본 우선순위를 바꾸고, 그에 따라 실질 우선순위를 재계산한 뒤 선점을 확인한다.

```

1. int
2. thread_get_priority (void)
3. {
4.     return thread_current ()->priority;
5. }

```

thread_get_priority()의 경우에는 thread_current()->priority를 반환하면 되므로 수정할 필요가 없다. 이미 priority 멤버가 실질 우선순위를 나타내도록 관리되기 때문이다.

C. priority donation 구현

priority donation을 처리하려면 thread의 기본 우선순위와 donate 받은 것까지 포함한 실질 우선순위를 구분해야 한다.

```

1. // threads/thread.c
2.
3. static void
4. init_thread (struct thread *t, const char *name, int priority){
5.     t->priority = priority;
6.
7.     t->base_priority = priority;
8.     t->waiting_lock = NULL;
9.     list_init (&t->donations);
10. }
11.

```

먼저 threads/thread.h의 struct thread에서 선언한 멤버인 base_priority(기본 우선순위를 나타냄), waiting_lock, donations를 초기화한다.

```

1. // threads/thread.c
2.
3. void
4. thread_recalculate_priority (struct thread *t)
5. {
6.     int max_priority = t->base_priority;
7.
8.     if (!list_empty(&t->donations))
9.     {
10.        struct list_elem *e;
11.        for (e = list_begin(&t->donations); e != list_end(&t->donations); e = list_next(e))
12.        {
13.            struct thread *donor = list_entry(e, struct thread, donation_elem);
14.            if (donor->priority > max_priority) {
15.                max_priority = donor->priority;
16.            }
17.        }
18.    }
19.

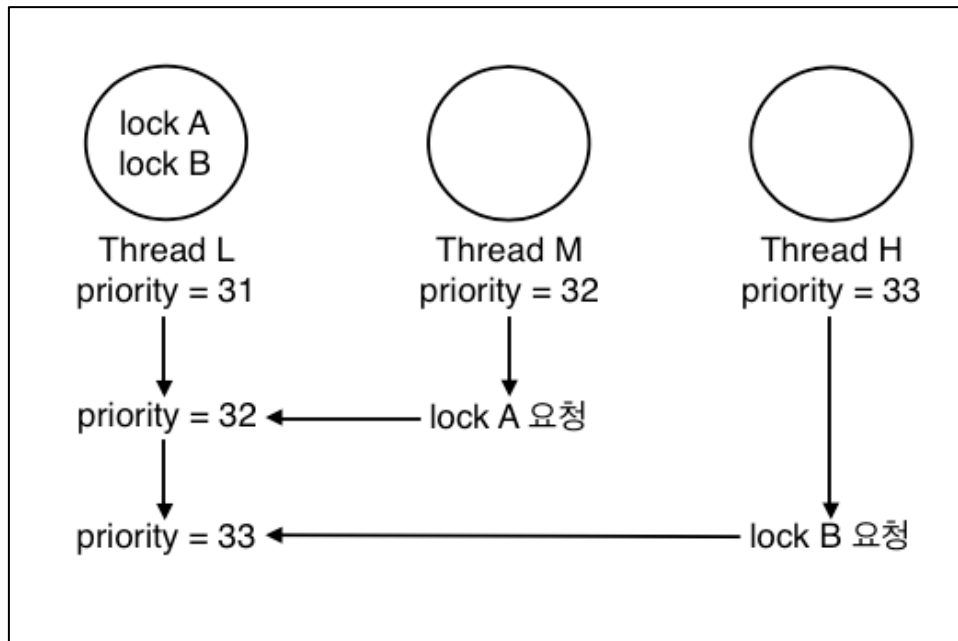
```

```

20. t->priority = max_priority;
21. }
22.

```

Priority donation 메커니즘에 따라 thread의 실행 우선순위를 재계산한다. 이 과정에서 기본 우선순위가 donate를 받은 실질 우선순위로 바뀌게 된다. 여러 thread가 donate한 priority를 종합하여, 최종적으로 반드시 가장 높은 priority를 가지도록 보장한다.

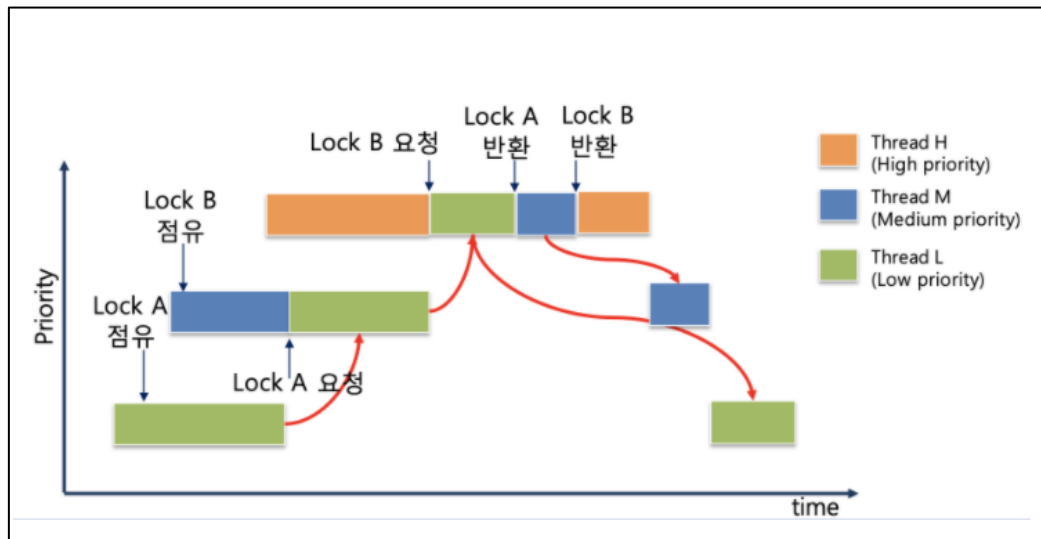


```

1. // threads/thread.c
2.
3. void
4. thread_donate_priority (struct thread *holder)
5. {
6.     if (holder == NULL) return;
7.
8.     struct thread *donor = thread_current();
9.
10.    list_push_back(&holder->donations, &donor->donation_elem);
11.
12.    while(holder) {
13.        thread_recalculate_priority(holder);
14.        if (holder->waiting_lock) {
15.            holder = holder->waiting_lock->holder;
16.        } else {
17.            break;
18.        }
19.    }
20. }
21.

```

현재 실행 중인 thread(기부자)의 우선순위를 lock을 점유하고 있는 thread(소유자)에게 donate한다. 이 과정에서 priority inversion 문제를 해결하기 위해, lock 대기 중인 여러 thread에 걸쳐 lock을 기다리는 thread가 없을 때까지 priority donation이 연쇄적으로 전파되는 구조를 채택했다. 이 과정에서 nested donation 문제가 해결된다.



```

1. // threads/synch.c
2.
3. void
4. lock_acquire (struct lock *lock)
5. {
6.     ASSERT (lock != NULL);
7.     ASSERT (!intr_context ());
8.     ASSERT (!lock_held_by_current_thread (lock));
9.
10.    struct thread *cur = thread_current();
11.
12.    if (lock->holder != NULL)
13.    {
14.        cur->waiting_lock = lock;
15.        thread_donate_priority(lock->holder);
16.    }
17.
18.    sema_down (&lock->semaphore);
19.    lock->holder = cur;
20.    cur->waiting_lock = NULL;
21. }

```

특정 lock이 다른 thread에 의해 소유되고 있으면, 현재 thread는 이 lock을 기다리고 있다고 표시하고 lock을 가진 thread에게 현재 thread priority를 donate한다.

```

1. // threads/thread.c
2.
3. void
4. thread_remove_donations_for_lock (struct lock *lock)
5. {
6.    struct thread *cur = thread_current();
7.    struct list_elem *e;
8.
9.    for (e = list_begin(&cur->donations); e != list_end(&cur->donations); )
10.    {
11.        struct thread *donor = list_entry(e, struct thread, donation_elem);
12.        if (donor->waiting_lock == lock) {
13.            e = list_remove(e);
14.        } else {
15.            e = list_next(e);
16.        }
17.    }
18. }

```

위의 함수는 현재 thread가 가지고 있는 priority donation 목록 중에서 특정 lock에 관련

된 donation을 제거하는 기능을 수행한다. Lock을 해제하거나 더 이상 donation이 필요 없을 때 해당 lock에 대한 donation 목록을 제거하는 용도로 사용된다.

```
1. // synch.c
2.
3. void
4. lock_release (struct lock *lock)
5. {
6.     ASSERT (lock != NULL);
7.     ASSERT (lock_held_by_current_thread (lock));
8.
9.     struct thread *cur = thread_current();
10.
11.     thread_remove_donations_for_lock(lock);
12.     thread_recalculate_priority(cur);
13.
14.     lock->holder = NULL;
15.     sema_up (&lock->semaphore);
16. }
17.
```

Lock_release는 현재 thread가 가지고 있는 lock을 해제하고, priority donation 상태를 갱신한다.

Pintos1 메뉴얼에서 다른 synchronization constructs에 대해서는 priority donation을 구현할 필요가 없다고 나와 있으므로, semaphore와 condition variable에 대해서는 priority donation을 따로 구현하지 않았다.

3) Advanced scheduler

2)에서 구현할 priority scheduler (priority를 고려한 scheduling 방식)의 경우, priority가 낮은 thread는 CPU를 차지하기 어려워 오랫동안 CPU에서 실행되지 못할 수 있다.

Pintos 1에서는 advanced scheduler – multilevel feedback queue scheduler (MLFQS)를 구현하려 한다. MLFQS는 각 priority별로 ready queue를 사용하여 priority를 자동적으로 관리한다. 다만, 여기서는 그 한 예시인 4.BSD scheduler를 구현하기 때문에 여러 Queue를 사용하는 대신, 세 개의 변수를 추가로 선언하여 동적으로 우선순위를 관리할 예정이다.

thread의 priority는 4 timer ticks 주기로 refresh된다. MLFQS를 구현하기 위해, 다음의 rule을 따라야 한다 (여기서는 priority donation을 고려하지 않는다):

- thread_set_priority() should not modify a priority
- thread_set_nice() updates priority
 - $priority = PRI_MAX - (recent_cpu / 4) - (nice * 2)$
 - $recent_cpu = (2 * load_avg) / (2 * load_avg + 1) * recent_cpu + nice$
 - $load_avg = (59/60) * load_avg + (1/60) * ready_threads$

* nice: thread가 다른 thread에게 CPU를 yield하는 정도를 나타내는 정수로, -20 ~ 20의 범위를 가진다. 값이 클수록 CPU를 잘 yield하며, 이때 해당 thread의 priority가 낮아져 실행이 뒤로 밀리게 된다. 반대로 값이 작을수록 해당 thread의 priority가 커진다.

* recent_cpu: thread가 최근에 CPU를 점유한 시간을 float 값으로 저장하며, 각 프로세스가 최근에 얼마나 CPU 시간을 할당 받았는지 측정하기 위해 사용된다. 최근의 CPU 사용량에 더 큰 가중치를 부여하여 스케줄링에 반영한다. 이 값이 클수록 최근에 더 많은 CPU를 사용했다는 의미이므로, 우선순위는 낮아진다.

* load_avg: 지난 1분 동안 ready state에 있는 thread의 평균 수를 추정한다. (ready_threads는 업데이트 시점에서 실행 중이거나 ready state에 있는 thread의 수 (idle 제외)를 의미한다) 이 값이 크다는 건 현재 부하가 크다는 거고, 작다는 건 부하가 작다는 것이다. 즉, 이 값이 크면 CPU 경쟁이 심해 우선순위가 천천히 증가하고, 작으면 우선순위가 빠르게 증가할 수 있다.

Initial Design:

MLFQS scheduler 구현에서의 핵심은 다음의 세 가지이다:

(1) pintos에서는 부동소수점 연산 지원이 없으므로, pintos manual을 참고하여 fixed-point macro 연산 파일을 만들어 효율적인 동적 우선순위 조절과 CPU 사용 시간 계산을 구현한다.

(2) nice, recent_cpu, load_avg라는 세 변수 기반으로 thread priority를 동적으로 관리한다.

(3) priority donation 관련 로직은 MLFQS 모드에서 무시하여 충돌을 방지한다.

코드 설계를 위해 디자인한 변수 및 함수 (+ 기존 스케레톤 코드에서 수정하려 하는 변수 및 함수)는 다음과 같다 (변수 및 함수는 Pintos 1 스케레톤 코드를 참고하여 거시적인 관점에서 고려하였다):

Variables:

- int nice – thread가 다른 thread에게 CPU를 yield하는 정도를 나타내는 정수이다.
- int recent_cpu – 해당 thread가 최근에 점유한 CPU 시간의 fixed-point 값을 나타낸다.
- static int load_avg – 지난 1분 동안 ready state에 있는 thread의 평균 수를 나타낸다.

Functions:

- thread_tick() - pintos manual의 reference guide B.4.4BSD scheduler를 참고하여, mlfqs flag 활성화 시 현재 thread recent_cpu 1 증가 / 매 4 tick마다 모든 thread의 priority 재계산 / 매 1초(tick 수 기준)마다 load_avg 및 모든 thread recent_cpu

재계산이 포함되도록 구현하려 한다.

- mlfqs_recalculate_priority() – priority 계산 공식에 따라 priority를 업데이트한다.
- mlfqs_recalculate_recent_cpu() – recent_cpu를 load_avg와 nice를 이용해 갱신하도록 한다.
- mlfqs_recalculate_load_avg() – load_avg를 현재 실행 가능한 thread 수 기반으로 갱신한다.
- thread_set_nice(), thread_get_nice(), thread_get_priority(), thread_get_recent_cpu(), thread_get_load_avg() – MLFQS interface 함수로, fixed-point 값을 적절히 변환해서 반환 및 설정한다.

* lock_acquire() 및 lock_release() 변형: thread_mlfqs flag 활성화 시 priority donation을 사용하지 않고 간단하게 semaphore만 사용하도록 구현할 예정이다.

3-1) Implementation:

Convert n to fixed point:	$n * f$
Convert x to integer (rounding toward zero):	x / f
Convert x to integer (rounding to nearest):	$(x + f / 2) / f$ if $x \geq 0$, $(x - f / 2) / f$ if $x \leq 0$.
Add x and y:	$x + y$
Subtract y from x:	$x - y$
Add x and n:	$x + n * f$
Subtract n from x:	$x - n * f$
Multiply x by y:	$((\text{int64_t}) x) * y / f$
Multiply x by n:	$x * n$
Divide x by y:	$((\text{int64_t}) x) * f / y$
Divide x by n:	x / n

```
1. // threads/fixed-point.h
2.
3. #ifndef THREADS_FIXED_POINT_H
4. #define THREADS_FIXED_POINT_H
5.
6. #define F (1 << 14)
7.
8. #define INT_TO_FP(n) ((n) * F)
9.
10. #define FP_TO_INT_ZERO(x) ((x) / F)
11.
```

```

12. #define FP_TO_INT_NEAREST(x) ((x) >= 0 ? ((x) + F / 2) / F : ((x) - F / 2) / F)
13.
14. #define FP_ADD(x, y) ((x) + (y))
15. #define FP_SUB(x, y) ((x) - (y))
16. #define FP_ADD_INT(x, n) ((x) + (n) * F)
17. #define FP_SUB_INT(x, n) ((x) - (n) * F)
18.
19. #define FP_MUL(x, y) (((int64_t)(x)) * (y) / F)
20. #define FP_MUL_INT(x, n) ((x) * (n))
21. #define FP_DIV(x, y) (((int64_t)(x)) * F / (y))
22. #define FP_DIV_INT(x, n) ((x) / (n))
23.
24. #endif
25.

```

Design에서 언급한 대로 fixed-point 연산을 수행하기 위해, 새롭게 fixed-point.h를 만들고 매크로를 사용했다. 코드는 핀토스 메뉴얼의 Reference Guide B.6.을 참고했다. 소수점 앞에 17bit, 뒤에 14bit가 있어 17.14 fixed-point number representation이라 불린다.

```

1. // threads/thread.h
2.
3. struct thread{
4.     int nice;
5.     int recent_cpu;
6. }
7.

```

```

1. // threads/thread.c
2.
3. static int load_avg;
4.

```

priority 조절을 위한 세 가지 변수 nice, recent_cpu, load_avg를 추가했다.

```

1. // threads/threads.c
2.
3. void
4. thread_init (void)
5. {
6.     load_avg = 0;
7. }
8.

```

이때 load_avg는 thread별이 아닌 전체 시스템에서 동일한 값을 쓰기 때문에, thread_init()에서 초기화해준다.

```

1. static void
2. init_thread (struct thread *t, const char *name, int priority)
3. {
4.     enum intr_level old_level;
5.
6.     ASSERT (t != NULL);
7.     ASSERT (PRI_MIN <= priority && priority <= PRI_MAX);
8.     ASSERT (name != NULL);
9.
10.    memset (t, 0, sizeof *t);
11.    t->status = THREAD_BLOCKED;
12.    strcpy (t->name, name, sizeof t->name);
13.    t->stack = (uint8_t *) t + PGSIZE;
14.    t->priority = priority;
15.
16.    t->nice = 0;
17.    t->recent_cpu = 0;
18.
19.    t->base_priority = priority;

```

```

20. t->waiting_lock = NULL;
21. list_init (&t->donations);
22.
23. t->magic = THREAD_MAGIC;
24.
25. old_level = intr_disable ();
26. list_push_back (&all_list, &t->allelem);
27. intr_set_level (old_level);
28. }
29.

```

nice, recent_cpu는 init_thread에서 초기화하여 주었다.

(thread_init(): 전체 초기화, init_thread(): thread 하나 초기화)

A. MLFQS scheduler 구현

```

1. // threads/threads.c
2.
3. void
4. thread_tick (void)
5. {
6.     struct thread *t = thread_current ();
7.
8.     /* Update statistics. */
9.     if (t == idle_thread)
10.         idle_ticks++;
11. #ifdef USERPROG
12.     else if (t->pagedir != NULL)
13.         user_ticks++;
14. #endif
15.     else
16.         kernel_ticks++;
17.
18.     if (thread_mlfqs)
19.     {
20.         /* 매 틱마다: 현재 스레드(idle 제외)의 recent_cpu 1 증가 */
21.         if (t != idle_thread)
22.             t->recent_cpu = FP_ADD_INT(t->recent_cpu, 1);
23.
24.         /* 매 4 틱마다: 모든 스레드의 priority 재계산 */
25.         if (timer_ticks() % TIME_SLICE == 0)
26.             thread_foreach(mlfqs_recalculate_priority, NULL);
27.
28.         /* 매 1초마다: load_avg 와 모든 스레드의 recent_cpu 재계산 */
29.         if (timer_ticks() % TIMER_FREQ == 0)
30.         {
31.             mlfqs_recalculate_load_avg();
32.             thread_foreach(mlfqs_recalculate_recent_cpu, NULL);
33.         }
34.     }
35.
36.     /* Enforce preemption. */
37.     if (++thread_ticks >= TIME_SLICE)
38.         intr_yield_on_return ();
39. }
40.

```

기존 코드에서 if (thread_mlfqs) 부분을 새롭게 추가했다. thread_mlfqs flag의 경우 pintos1 스케레톤 코드에 포함된 변수로, 이 변수를 사용하여 mlfqs scheduler 사용 여부를 판단했다. Pintos 메뉴얼의 Reference Guide B.4.4BSD Scheduler를 참고하면,

- 매 tick마다 현재 스레드의 recent_cpu를 1씩 증가
- 매 4 tick마다 모든 스레드에 대해 우선순위 재계산
- 매 1초마다 load_avg와 모든 스레드의 recent_cpu 재계산

다음과 같은 내용이 나와 있어 이를 적용했다.

```

1. // threads/threads.c
2.
3. static void
4. mlfqs_recalculate_priority (struct thread *t, void *aux UNUSED)
5. {
6.     if (t != idle_thread)
7.     {
8.         int term1 = FP_TO_INT_NEAREST (FP_DIV_INT (t->recent_cpu, 4));
9.         int term2 = t->nice * 2;
10.        int new_priority = PRI_MAX - term1 - term2;
11.
12.        if (new_priority > PRI_MAX)
13.            t->priority = PRI_MAX;
14.        else if (new_priority < PRI_MIN)
15.            t->priority = PRI_MIN;
16.        else
17.            t->priority = new_priority;
18.    }
19. }
20.
21. static void
22. mlfqs_recalculate_recent_cpu (struct thread *t, void *aux UNUSED)
23. {
24.     if (t != idle_thread)
25.     {
26.         int load_x_2 = FP_MUL_INT (load_avg, 2);
27.         int decay_coeff = FP_DIV (load_x_2, FP_ADD_INT (load_x_2, 1));
28.
29.         t->recent_cpu = FP_ADD_INT (FP_MUL (decay_coeff, t->recent_cpu), t->nice);
30.     }
31. }
32.
33. static void
34. mlfqs_recalculate_load_avg (void)
35. {
36.     int ready_threads = list_size (&ready_list);
37.     if (thread_current () != idle_thread)
38.         ready_threads++;
39.
40.     int term1 = FP_MUL (FP_DIV_INT (INT_TO_FP (59), 60), load_avg);
41.     int term2 = FP_MUL_INT (FP_DIV_INT (INT_TO_FP (1), 60), ready_threads);
42.
43.     load_avg = FP_ADD (term1, term2);
44. }
45.

```

위의 코드는 MLFQS 헬퍼 함수들이다. 기존 priority scheduler와 달리 donation logic도 제외해야 하고, priority, recent_cpu, 그리고 load_avg는 재계산이 필요한 작업이므로, 공식을 참고해 다음과 같은 헬퍼 함수를 추가했다.

```

1. // threads/thread.c
2.
3. /* Returns the current thread's priority. */
4. int
5. thread_get_priority (void)
6. {
7.     return thread_current ()->priority;

```

```

8. }
9.
10. /* Sets the current thread's nice value to NICE. */
11. void
12. thread_set_nice (int nice)
13. {
14.     struct thread *cur = thread_current();
15.     cur->nice = nice;
16.     mlfqs_recalculate_priority(cur, NULL);
17.     test_max_priority();
18. }
19.
20. /* Returns the current thread's nice value. */
21. int
22. thread_get_nice (void)
23. {
24.     return thread_current()->nice;
25. }
26.
27. /* Returns 100 times the system load average. */
28. int
29. thread_get_load_avg (void)
30. {
31.     return FP_TO_INT_NEAREST(FP_MUL_INT(load_avg, 100));
32. }
33.
34. /* Returns 100 times the current thread's recent_cpu value. */
35. int
36. thread_get_recent_cpu (void)
37. {
38.     return FP_TO_INT_NEAREST(FP_MUL_INT(thread_current()->recent_cpu, 100));
39. }
40.

```

MLFQS scheduler의 인터페이스 함수들은 위와 같이 구현했다. 이때 thread_set_nice()의 경우 새롭게 설정된 nice 값을 바탕으로 mlfqs의 priority가 재계산된다.

B. donation 비활성화

```

1. // threads/synch.c
2.
3. void
4. lock_acquire (struct lock *lock)
5. {
6.     if (thread_mlfqs) {
7.         sema_down(&lock->semaphore);
8.         lock->holder = thread_current();
9.         return;
10.    }
11. }
12.
13. void
14. lock_release (struct lock *lock)
15. {
16.     if (thread_mlfqs) {
17.         lock->holder = NULL;
18.         sema_up(&lock->semaphore);
19.         return;
20.    }
21. }
22.

```

2)의 priority scheduling에서와 다르게, 3)에서는 donation을 고려하지 않으므로 위의 logic을 donation을 담당하는 동기화 함수에도 추가해주었다.

4. Discussion

A. What is different from your design report and why did you take this change?

1) Alarm Clock

초기 설계와 실제 구현은 동일하다. Busy Waiting에서 Alarm Clock으로 전환을 위해 Sleep list를 도입하였다. 또한, 효율을 위해 항상 정렬되도록 하였고, timer_interrupt()를 수정하여 Alarm Clock 효과를 내도록 하였다.

2) Priority Scheduler

전반적으로는 비슷하다. 하지만, Design report에서는 텍스트로만 설명했던 정렬 부분이 구체적으로 구현되었다. sema_down(), cond_wait()에서 우선순위 순으로 삽입되는 게 이제 list_insert_ordered(), sema_up(), cond_signal()에서 우선순위로 정렬되는 게 list_sort()를 통해 구현되었다.

3) MLFQS

마찬가지로 전반적으로 유사하다. Lock_acquire, Lock Release에 수정이 필요했는데, 이는 Priority Donation Logic을 무시하기 위함이었다. 원래는 semaphore 사용하도록 구현하는 말만 있었는데, 실제 mlfqs flag 활성화 시 각각 sema_down(), sema_up() 호출하도록 구현하였다.

B. What have you learned?

이번 프로젝트를 통해 thread라는 큰 틀에서 스케줄링과 동기화에 대한 깊이 있는 이해를 얻을 수 있었다. 기존의 실시간 시스템에 유리한 Busy Waiting 방식의 timer_sleep()을 Sleep/Wake-up 기반의 Alarm Clock으로 개선하여 thread를 효율적으로 관리할 수 있었다. 또한, Round-Robin(RR) Scheduler를 Priority Scheduler로 바꾸면서 단순히 thread를 순서대로 실행하는 것이 아닌, 중요도에 맞게 할당을 조절하는 스케줄링의 중요성을 배웠다. 그리고 여러 thread가 공유 자원에 접근할 때 발생하는 Race Condition을 해결하기 위해 3가지 Synchronization tool인 Semaphore, Lock, Condition Variable의 실제 구현을 볼 수 있었다. 특히, Priority Donation 메커니즘을 구현하면서 복잡한 동기화 상황을 이해할 수 있었다. 또한 MLFQS는 이전에 들어 봤었는데, nice, recent_cpu, load_avg와 같은 변수를 활용해 동적으로 조절하는지는 몰랐고 이번 기회에 배울 수 있었다. Pintos에서는 fixed-point 연산을 지원하지 않아 직접 적용해보는 과정도 도움이 되었다.

C. What are the limitations of your implementation?

이번 Pintos 1 구현은 요구사항과 테스트를 모두 통과한 것으로 확인했지만, 몇 가지 한계점이 존재한다. 현재 구현된 17.14 fixed-point representation 자체가 정밀도에 한계가

있어 계산 결과가 오차가 발생할 가능성이 존재한다. 특히, 계산에서 소수점 이하 미세한 값들은 반올림 처리되어 실제 값과 약간의 차이가 누적될 가능성이 있다. 그리고 현재 logic은 Nested Donation, Multiple Donation을 처리할 수 있도록 구현되었지만, 더 복잡한 자원 경쟁 상황에서는 예상치 못한 버그가 발생할 수 있다. 또한 MLFQS의 Priority Donation Logic 결여: 현재 요구사항을 반영하느라, MLFQS에서는 Priority Donation Logic 이 제외되어 있다. 물론 구현 시 복잡성도 높아지고, 권장되지 않는다는 말도 있지만, 현재 구현에서는 이 기능이 사용 불가함은 맞는 말이다.

D. Test Result

```
pass tests/threads/mlfqs-block
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-donate-one
pass tests/threads/priority-donate-multiple
pass tests/threads/priority-donate-multiple2
pass tests/threads/priority-donate-nest
pass tests/threads/priority-donate-sema
pass tests/threads/priority-donate-lower
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-condvar
pass tests/threads/priority-donate-chain
pass tests/threads/mlfqs-load-1
pass tests/threads/mlfqs-load-60
pass tests/threads/mlfqs-load-avg
pass tests/threads/mlfqs-recent-1
pass tests/threads/mlfqs-fair-2
pass tests/threads/mlfqs-fair-20
pass tests/threads/mlfqs-nice-2
pass tests/threads/mlfqs-nice-10
pass tests/threads/mlfqs-block
All 27 tests passed.
make[1]: Leaving directory '/home/stu12/pintos/src/threads/build'
stu12@cse-edu:~/pintos/src/threads$ git branch
  main
  master
  pintos1
* project1
stu12@cse-edu:~/pintos/src/threads$ |
```

제출 서버의 project1 branch에서 test 결과, 27개의 테스트 모두 통과함을 확인하였다.